Buổi 12

Thiết kế chương trình hướng đối tượng

Học phần: Lập trình hướng đối tượng

Nội dung bài học

1. Quy trình phát triển phần mềm hướng đối tượng

1.1 Các giai đoạn phát triển hệ thống
1.2 Năm bước thiết kế một chương trình hướng đối tượng

2. Phân tích bài toán: tìm lớp, thuộc tính, phương thức

2.1 Đặc tả bài toán quản lý thư viện
2.2 Gán phương thức cho lớp sở hữu dữ liệu

3. Thiết kế quan hệ giữa các lớp

3.1 Ba loại quan hệ và ký hiệu UML
3.2 Chọn kế thừa hay thành phần?

4. Cài đặt ví dụ: chương trình quản lý thư viện

4.1 Lớp trừu tượng TaiLieu và các lớp dẫn xuất
4.2 Lớp BanDoc và PhieuMuon
4.3 Lớp ThuVien – quản lý và nghiệp vụ mượn trả
4.4 In danh sách, lưu file và chương trình chính
4.5 Mở rộng chương trình: nạp lại dữ liệu từ file

5. Mini-project và tổng kết môn học

5.1 Danh mục kiểm tra một thiết kế tốt
5.2 Yêu cầu mini-project
5.3 Sản phẩm nộp và tiêu chí đánh giá
5.4 Tổng kết môn học
5.5 Bài thực hành buổi 12 – Thiết kế và cài đặt mini-project
5.6 Câu hỏi và bài tập

1.1 Các giai đoạn phát triển hệ thống

Giai đoạn	Câu hỏi chính	Sản phẩm
Phân tích (OOA)	Hệ thống **làm gì**? Có những đối tượng nào trong bài toán?	Đặc tả yêu cầu, danh sách lớp/đối tượng, ca sử dụng
Thiết kế (OOD)	Hệ thống **làm như thế nào**? Lớp có thuộc tính, phương thức, quan hệ gì?	Sơ đồ lớp UML, giao diện các lớp
Lập trình (OOP)	Cài đặt các lớp bằng ngôn ngữ cụ thể (C++)	Mã nguồn .h / .cpp
Kiểm thử	Từng lớp chạy đúng? Các lớp phối hợp đúng?	Bộ ca kiểm thử, báo cáo lỗi
Bảo trì	Sửa lỗi, mở rộng chức năng	Phiên bản mới

Tư tưởng cốt lõi

Mô hình hóa thế giới thực thành **các đối tượng cộng tác** với nhau bằng cách gửi thông điệp (gọi phương thức). Mỗi lớp chịu **một trách nhiệm rõ ràng**, che giấu dữ liệu của mình và chỉ công bố giao diện cần thiết.

- Quá trình **lặp**: thiết kế → cài đặt một phần → kiểm thử → điều chỉnh thiết kế. Không cần (và không thể) thiết kế hoàn hảo ngay lần đầu.
- So với hướng thủ tục (chia nhỏ **chức năng**), hướng đối tượng chia nhỏ theo **dữ liệu + hành vi** ⇒ dễ tái sử dụng (kế thừa) và dễ mở rộng (đa hình).

1.2 Năm bước thiết kế một chương trình hướng đối tượng

Các bước thực hiện

- **Bước 1 – Xác định lớp:** đọc đặc tả, gạch dưới các **danh từ** ⇒ ứng viên lớp; loại bỏ danh từ trùng nghĩa, quá mơ hồ, hoặc chỉ là một giá trị đơn giản (thuộc tính).
- **Bước 2 – Xác định thuộc tính:** dữ liệu mà mỗi đối tượng cần **ghi nhớ** (tính từ, danh từ sở hữu: "mã sách", "số lượng").
- **Bước 3 – Xác định phương thức:** gạch dưới các **động từ** ⇒ hành vi; gán mỗi hành vi cho lớp **sở hữu dữ liệu** liên quan.
- **Bước 4 – Xác định quan hệ:** kế thừa ("là một"), kết tập/thành phần ("có một"), sử dụng ("dùng"); tìm phần chung để đưa lên lớp cơ sở (trừu tượng).
- **Bước 5 – Cài đặt và kiểm thử:** viết khai báo lớp (.h), cài đặt (.cpp), chương trình chính thử từng chức năng; kiểm tra rò rỉ bộ nhớ.

Nguyên tắc thiết kế cần nhớ

- **Đóng gói:** dữ liệu private/protected, truy xuất qua phương thức; không để lớp khác sửa trực tiếp.
- **Một trách nhiệm:** lớp làm quá nhiều việc ⇒ tách lớp. Lớp gần như rỗng ⇒ có thể chỉ là thuộc tính.
- **Mở để mở rộng, đóng để sửa đổi:** thêm loại mới bằng **lớp dẫn xuất + hàm ảo**, không phải sửa `switch` khắp nơi.

2.1 Đặc tả bài toán quản lý thư viện

Đặc tả (gạch dưới danh từ và in nghiêng động từ khi phân tích)

Thư viện quản lý các tài liệu gồm sách và tạp chí. Mỗi tài liệu có mã, tên, số lượng còn trong kho. Sách có thêm tác giả, được **mượn** về nhà tối đa 14 ngày. Tạp chí có số kỳ và **chỉ được đọc tại chỗ**. Bạn đọc có mã, họ tên, được **mượn** tối đa 5 tài liệu cùng lúc. Mỗi lần mượn lập một phiếu mượn ghi bạn đọc, tài liệu, ngày mượn. Khi **trả**, nếu quá hạn thì **tính tiền phạt** 5000 đồng mỗi ngày trễ. Thủ thư có thể **thêm** tài liệu, **tìm** tài liệu theo mã, **in** danh sách và **lưu** danh sách ra file.

Danh từ	Quyết định	Lý do
tài liệu	**Lớp trừu tượng** TaiLieu	Khái niệm chung của sách và tạp chí, không tạo trực tiếp
sách, tạp chí	**Lớp dẫn xuất** Sach, TapChi	"Sách **là một** tài liệu"; có dữ liệu và luật mượn riêng
bạn đọc	**Lớp** BanDoc	Có dữ liệu (mã, tên, số đang mượn) và hành vi (mượn, trả)
phiếu mượn	**Lớp** PhieuMuon	Liên kết bạn đọc – tài liệu – ngày; tính tiền phạt
thư viện	**Lớp** ThuVien	Quản lý tập hợp tài liệu, bạn đọc, phiếu mượn
mã, tên, số lượng, tác giả, số kỳ, ngày mượn	**Thuộc tính**	Giá trị đơn giản, không có hành vi riêng
14 ngày, 5 tài liệu, 5000 đồng	**Hằng / quy tắc**	Cài trong phương thức (hàm ảo SoNgayMuonToiDa…)
thủ thư	**Loại bỏ**	Là người dùng chương trình, không cần lưu dữ liệu

2.2 Gán phương thức cho lớp sở hữu dữ liệu

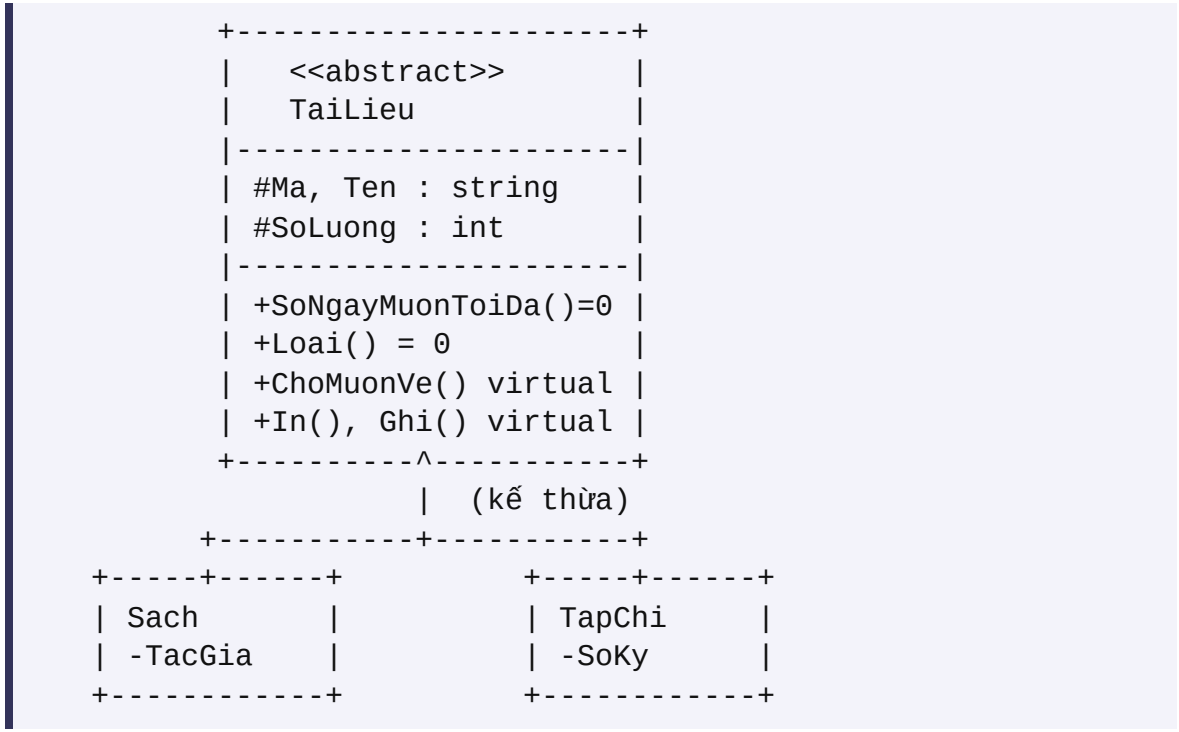
Động từ trong đặc tả	Phương thức	Thuộc lớp	Vì sao
mượn về nhà tối đa 14 ngày	`SoNgayMuonToiDa()`	TaiLieu (ảo thuần), Sach, TapChi	Mỗi loại một luật ⇒ đa hình
chỉ đọc tại chỗ	`ChoMuonVe()`	TaiLieu (ảo, mặc định true), TapChi ghi đè false	Ngoại lệ của một lớp con
còn trong kho? giảm/tăng	`ConSach()`, `Giam()`, `Tang()`	TaiLieu	Thao tác trên SoLuong – dữ liệu của TaiLieu
mượn tối đa 5	`CoTheMuon()`, `Muon()`, `Tra()`	BanDoc	Thao tác trên DangMuon
tính tiền phạt	`TienPhat(ngayTra)`	PhieuMuon	Phiếu biết ngày mượn và tài liệu
thêm, tìm, mượn, trả, in, lưu	`ThemTaiLieu`, `TimTaiLieu`, `ChoMuon`, `NhanTra`, `InDanhSach`, `Luu`	ThuVien	Cần truy cập toàn bộ tập hợp

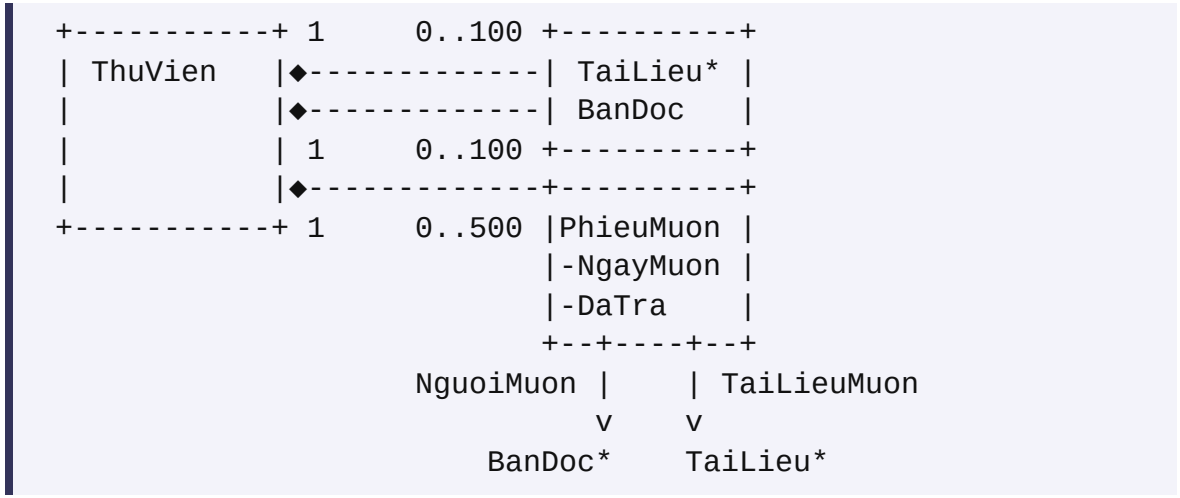
Sai lầm thường gặp

- Đặt `TienPhat` trong ThuVien và viết `if (loai == "Sach") … else if (loai == "TapChi") …` ⇒ thêm loại "Luận văn" phải sửa mọi chỗ. Đúng: hỏi **chính đối tượng** qua hàm ảo `SoNgayMuonToiDa()`.
- Để thuộc tính public cho "tiện": ThuVien sửa thẳng `SoLuong = -1` ⇒ phá vỡ ràng buộc. Đúng: chỉ qua `Giam()` sau khi đã kiểm tra `ConSach()`.

3.1 Ba loại quan hệ và ký hiệu UML

Quan hệ	Ý nghĩa	UML	Cài đặt C++	Ví dụ thư viện
Kế thừa (tổng quát hóa)	B **là một** A	Mũi tên tam giác rỗng B ▷ A	`class B : public A`	Sach, TapChi ▷ TaiLieu
Thành phần (composition)	A **sở hữu** B, B sống chết theo A	Hình thoi đặc ◆ phía A	Thành viên đối tượng, hoặc con trỏ được A `delete`	ThuVien ◆ BanDoc, PhieuMuon; ThuVien sở hữu các TaiLieu* (delete trong hàm hủy)
Kết tập / liên kết	A **tham chiếu** B, B tồn tại độc lập	Hình thoi rỗng ◇ hoặc đường thẳng	Con trỏ/tham chiếu, **không** delete	PhieuMuon → BanDoc, TaiLieu
Sử dụng (phụ thuộc)	A **dùng** B tạm thời	Mũi tên nét đứt	Tham số hàm, biến cục bộ	TaiLieu::In(ostream &)





- Bội số: 0..100 tài liệu; phiếu chỉ **trỏ tới** (không delete).

3.2 Chọn kế thừa hay thành phần?

Kiểm tra "là một" / "có một"

- "Sách **là một** tài liệu" – đúng mọi lúc ⇒ **kế thừa** công khai.
- "Thư viện **có** các tài liệu" ⇒ **thành phần**. Viết `class ThuVien : public TaiLieu` là **sai**: thư viện không phải tài liệu.
- "Bạn đọc **mượn** tài liệu" ⇒ quan hệ thông qua lớp trung gian **PhieuMuon** (lưu thêm dữ liệu của chính quan hệ: ngày mượn, đã trả).

- **Tìm lớp cơ sở chung:** Sach và TapChi cùng có mã, tên, số lượng, in, ghi file ⇒ đưa lên TaiLieu. Phần khác nhau (luật mượn) ⇒ hàm **ảo thuần**.
- **Lớp trừu tượng** ngăn tạo `TaiLieu t;` vô nghĩa và buộc mọi lớp con phải định nghĩa luật mượn.
- **Hàm hủy ảo** trong TaiLieu: bắt buộc vì ThuVien `delete DS[i]` qua con trỏ lớp cơ sở (buổi 10).

Mảng con trỏ đa hình

```
TaiLieu *DS[100];          // KHÔNG dùng TaiLieu DS[100]
DS[0] = new Sach(...);     // lưu được mọi lớp con
DS[1] = new TapChi(...);
DS[i]->SoNgayMuonToiDa();  // gọi đúng hàm của lớp con
```

- Mảng đối tượng `TaiLieu DS[100]` không biên dịch được (lớp trừu tượng) và nếu không trừu tượng thì sẽ bị **cắt lát** (slicing).
- ThuVien sở hữu các đối tượng ⇒ **cấm sao chép** (khai báo private constructor sao chép và `operator=`, không định nghĩa) để tránh delete hai lần – một cách áp dụng Quy tắc ba.

4.1 Lớp trừu tượng TaiLieu và các lớp dẫn xuất

```
class TaiLieu                       // lớp trừu tượng
{  public:
   TaiLieu(const string &ma, const string &ten, int sl)
      : Ma(ma), Ten(ten), SoLuong(sl) {}
   virtual ~TaiLieu() {}
   virtual int SoNgayMuonToiDa() const = 0;
   virtual string Loai() const = 0;
   virtual bool ChoMuonVe() const { return true; }
   bool ConSach() const { return SoLuong > 0; }
   void Giam() { SoLuong--; }
   void Tang() { SoLuong++; }
   const string &GetMa() const { return Ma; }
   virtual void In(ostream &os) const
   {  os << left << setw(6) << Ma << setw(22) << Ten
         << setw(8) << Loai() << right << setw(3) << SoLuong;
}
   virtual void Ghi(ostream &os) const
   {  os << Loai() << ';' << Ma << ';' << Ten << ';' <<
SoLuong; }
   protected:
   string Ma, Ten;  int SoLuong;
};
```

```
class Sach : public TaiLieu
{  public:
   Sach(const string &ma, const string &ten, int sl,
        const string &tg) : TaiLieu(ma, ten, sl), TacGia(tg)
{}
   int SoNgayMuonToiDa() const override { return 14; }
   string Loai() const override { return "Sach"; }
   void Ghi(ostream &os) const override
   {  TaiLieu::Ghi(os);  os << ';' << TacGia;  }
   private:  string TacGia;
};
class TapChi : public TaiLieu
{  public:
   TapChi(const string &ma, const string &ten, int sl, int
ky)
      : TaiLieu(ma, ten, sl), SoKy(ky) {}
   int SoNgayMuonToiDa() const override { return 0; }
   string Loai() const override { return "TapChi"; }
   bool ChoMuonVe() const override { return false; }
   void Ghi(ostream &os) const override
   {  TaiLieu::Ghi(os);  os << ';' << SoKy;  }
   private:  int SoKy;
};
```

- `Ghi` của lớp con **gọi lại** `TaiLieu::Ghi` rồi ghi thêm phần riêng – tái sử dụng mã lớp cha.
- `In(ostream &)` dùng được cho cout và file (buổi 11).

4.2 Lớp BanDoc và PhieuMuon

```
class BanDoc
{  public:
   BanDoc(const string &ma = "", const string &ten = "")
      : Ma(ma), Ten(ten), DangMuon(0) {}
   bool CoTheMuon() const { return DangMuon < 5; }
   void Muon() { DangMuon++; }
   void Tra()  { DangMuon--; }
   const string &GetMa() const  { return Ma; }
   const string &GetTen() const { return Ten; }
   private:
   string Ma, Ten;  int DangMuon;
};
```

- Tham số mặc định ⇒ BanDoc có **constructor mặc định**, nên khai báo được mảng `BanDoc BD[100]` trong ThuVien.
- Quy tắc "tối đa 5" nằm **trong** BanDoc – ThuVien chỉ hỏi `CoTheMuon()`.

```
class PhieuMuon
{  public:
   PhieuMuon(BanDoc *bd = NULL, TaiLieu *tl = NULL,
             int ngay = 0)
      : NguoiMuon(bd), TaiLieuMuon(tl),
        NgayMuon(ngay), DaTra(false) {}
   long TienPhat(int ngayTra) const
   {  int tre = ngayTra - NgayMuon
              - TaiLieuMuon->SoNgayMuonToiDa();  // đa hình
      return tre > 0 ? tre * 5000L : 0;  }
   BanDoc  *NguoiMuon;   TaiLieu *TaiLieuMuon;
   int NgayMuon;         bool DaTra;
};
```

Phân tích TienPhat

- Mượn Sach ngày 1, trả ngày 20: trễ = 20 − 1 − 14 = **5** ngày ⇒ phạt **25000**.
- Mượn ngày 3, trả ngày 10: trễ = 10 − 3 − 14 = −7 ≤ 0 ⇒ phạt **0**.
- Thêm loại "Luận văn" 7 ngày: chỉ viết lớp `LuanVan` ghi đè `SoNgayMuonToiDa()`, **không sửa** PhieuMuon.

4.3 Lớp ThuVien – quản lý và nghiệp vụ mượn trả

```
class ThuVien
{  public:
   ThuVien() : SoTL(0), SoBD(0), SoPM(0) {}
   ~ThuVien()
   {  for (int i = 0; i < SoTL; i++) delete DS[i];  }
   void ThemTaiLieu(TaiLieu *t)
   {  if (SoTL < 100) DS[SoTL++] = t;  }
   void ThemBanDoc(const BanDoc &b)
   {  if (SoBD < 100) BD[SoBD++] = b;  }
   TaiLieu *TimTaiLieu(const string &ma)
   {  for (int i = 0; i < SoTL; i++)
         if (DS[i]->GetMa() == ma) return DS[i];
      return NULL;  }
   BanDoc *TimBanDoc(const string &ma);   // tương tự
   string ChoMuon(const string &maBD,
                  const string &maTL, int ngay);
   long NhanTra(const string &maBD,
                const string &maTL, int ngay);
   void InDanhSach() const;
   bool Luu(const string &tenFile) const;
   private:
   TaiLieu *DS[100];  int SoTL;
   BanDoc BD[100];    int SoBD;
   PhieuMuon PM[500]; int SoPM;
   ThuVien(const ThuVien &);              // cấm sao chép
   ThuVien &operator=(const ThuVien &);
};
```

```
string ThuVien::ChoMuon(const string &maBD,
                         const string &maTL, int ngay)
{  BanDoc *b = TimBanDoc(maBD);
   TaiLieu *t = TimTaiLieu(maTL);
   if (!b || !t)          return "Khong tim thay";
   if (!t->ChoMuonVe())   return "Chi doc tai cho";
   if (!t->ConSach())     return "Het tai lieu";
   if (!b->CoTheMuon())   return "Da muon toi da";
   t->Giam();  b->Muon();
   PM[SoPM++] = PhieuMuon(b, t, ngay);
   return "OK";
}
long ThuVien::NhanTra(const string &maBD,
                      const string &maTL, int ngay)
{  for (int i = 0; i < SoPM; i++)
      if (!PM[i].DaTra
          && PM[i].NguoiMuon->GetMa() == maBD
          && PM[i].TaiLieuMuon->GetMa() == maTL)
      {  PM[i].DaTra = true;
         PM[i].TaiLieuMuon->Tang();
         PM[i].NguoiMuon->Tra();
         return PM[i].TienPhat(ngay);  }
   return -1;                       // không có phiếu
}
```

- ThuVien **điều phối**: mọi quy tắc nghiệp vụ được hỏi từ đối tượng sở hữu dữ liệu.

4.4 In danh sách, lưu file và chương trình chính

```
void ThuVien::InDanhSach() const
{  for (int i = 0; i < SoTL; i++)
   {  DS[i]->In(cout);  cout << endl;  }  }
bool ThuVien::Luu(const string &tenFile) const
{  ofstream f(tenFile);
   if (!f) return false;
   for (int i = 0; i < SoTL; i++)
   {  DS[i]->Ghi(f);  f << '\n';  }   // đa hình
   return true;  }
int main()
{  ThuVien tv;
   tv.ThemTaiLieu(new Sach("S01", "Lap trinh C++", 2, "Le My
Hanh"));
   tv.ThemTaiLieu(new Sach("S02", "Cau truc du lieu", 1,
"Tran Nhi"));
   tv.ThemTaiLieu(new TapChi("T01", "Tin hoc ngay nay", 3,
125));
   tv.ThemBanDoc(BanDoc("BD1", "An"));
   tv.ThemBanDoc(BanDoc("BD2", "Binh"));
   cout << tv.ChoMuon("BD1", "S02", 1) << endl;
   cout << tv.ChoMuon("BD2", "S02", 2) << endl;
   cout << tv.ChoMuon("BD2", "T01", 2) << endl;
   cout << tv.ChoMuon("BD2", "S01", 3) << endl;
   tv.InDanhSach();
   cout << "Phat: " << tv.NhanTra("BD1", "S02", 20) << endl;
   cout << "Phat: " << tv.NhanTra("BD2", "S01", 10) << endl;
   cout << tv.ChoMuon("BD2", "S02", 21) << endl;
   cout << (tv.Luu("thuvien.txt") ? "Da luu" : "Loi luu");
}
```

```
OK
Het tai lieu
Chi doc tai cho
OK
S01   Lap trinh C++          Sach      1
S02   Cau truc du lieu       Sach      0
T01   Tin hoc ngay nay       TapChi    3
Phat: 25000
Phat: 0
OK
Da luu
```

- Tập tin `thuvien.txt` sau khi chạy:

```
Sach;S01;Lap trinh C++;2;Le My Hanh
Sach;S02;Cau truc du lieu;0;Tran Nhi
TapChi;T01;Tin hoc ngay nay;3;125
```

- Giải thích: S02 chỉ có 1 cuốn ⇒ BD2 mượn lần 2 "Het tai lieu"; tạp chí bị từ chối; sau khi BD1 trả S02 thì BD2 mượn được (dòng OK cuối) ⇒ S02 còn 0.
- Đã biên dịch với `-fsanitize=address`: **không rò rỉ bộ nhớ** nhờ hàm hủy ThuVien.

4.5 Mở rộng chương trình: nạp lại dữ liệu từ file

```
bool ThuVien::Nap(const string &tenFile)
{  ifstream f(tenFile);
   if (!f) return false;
   string dong;
   while (getline(f, dong))
   {  stringstream ss(dong);
      string loai, ma, ten, sl, them;
      getline(ss, loai, ';');  getline(ss, ma, ';');
      getline(ss, ten, ';');   getline(ss, sl, ';');
      getline(ss, them);
      if (loai == "Sach")
         ThemTaiLieu(new Sach(ma, ten, stoi(sl), them));
      else if (loai == "TapChi")
         ThemTaiLieu(new TapChi(ma, ten, stoi(sl),
stoi(them)));
   }
   return true;
}
```

- Đây là **hàm nhà máy** (factory): là nơi **duy nhất** cần biết tên các lớp con để tạo đối tượng từ chuỗi "Sach"/"TapChi". Mọi chỗ khác chỉ làm việc với `TaiLieu *`.
- `getline(ss, x, ';')` tách trường theo dấu phân cách; `stoi` đổi chuỗi sang số (buổi 11).

Hướng mở rộng cho mini-project

- Lưu/nạp **bạn đọc** và **phiếu mượn** (phiếu lưu mã bạn đọc, mã tài liệu, ngày, trạng thái; khi nạp dùng `TimBanDoc`/`TimTaiLieu` để khôi phục con trỏ).
- Thay mảng cố định bằng lớp template `Array<T>` (buổi 11) hoặc `vector<TaiLieu*>`.
- Đa năng hóa `operator<<` cho TaiLieu bằng cách gọi hàm ảo `In` – in đúng mọi lớp con.
- Menu điều khiển: 1-Thêm, 2-Tìm, 3-Mượn, 4-Trả, 5-In, 6-Lưu, 0-Thoát.

5.1 Danh mục kiểm tra một thiết kế tốt

Tiêu chí	Câu hỏi tự kiểm tra	Áp dụng trong ví dụ
Đóng gói	Mọi thuộc tính đều private/protected?	SoLuong chỉ đổi qua Giam/Tang
Một trách nhiệm	Mô tả được lớp bằng một câu?	PhieuMuon: "ghi nhận một lần mượn và tính phạt"
Kế thừa đúng	Quan hệ "là một" luôn đúng?	Sach là TaiLieu; ThuVien **không** kế thừa TaiLieu
Đa hình thay switch	Có chuỗi `if (loai == …)` lặp lại không?	Chỉ còn ở hàm nạp (factory)
Quản lý bộ nhớ	Ai new thì ai delete? Có hàm hủy ảo? Quy tắc ba?	ThuVien delete; ~TaiLieu ảo; cấm sao chép
const đúng chỗ	Hàm không sửa đối tượng đã có const?	ConSach, In, Ghi, TienPhat là const
Giao diện tối thiểu	Có hàm public thừa, lộ cấu trúc bên trong?	Không trả về mảng DS ra ngoài
Kiểm thử	Đã thử biên, lỗi, trường hợp đặc biệt?	Hết sách, tạp chí, trả trễ, trả đúng hạn

Câu hỏi nhanh

- Muốn thêm "Báo" chỉ đọc tại chỗ, phải sửa những lớp nào? → Chỉ **thêm** lớp `Bao : public TaiLieu` và một nhánh trong hàm nạp; ThuVien, PhieuMuon **giữ nguyên**.

5.2 Yêu cầu mini-project

Đề tài (chọn một, nhóm 2–3 sinh viên)

- **Quản lý thư viện** (mở rộng ví dụ trên: thêm luận văn, lưu bạn đọc và phiếu mượn, thống kê quá hạn).
- **Quản lý bán hàng:** Hàng hóa (trừu tượng) → Thực phẩm (hạn dùng, giảm giá khi gần hết hạn), Điện máy (bảo hành), Sành sứ; Hóa đơn gồm nhiều Dòng hóa đơn; Khách hàng; tính tổng tiền, VAT.
- **Quản lý nhân sự – lương:** NhanVien (trừu tượng, hàm ảo `TinhLuong()`) → Biên chế, Hợp đồng, Bán thời gian; Phòng ban chứa nhân viên.
- **Quản lý điểm sinh viên:** SinhVien → Chính quy, Liên thông; MonHoc, KetQua; tính điểm TB, xếp loại, in bảng điểm.

Yêu cầu kỹ thuật bắt buộc

- Ít nhất **5 lớp**, có **lớp trừu tượng** và **≥ 2 lớp dẫn xuất** dùng **hàm ảo**; có quan hệ **thành phần**.
- Có **constructor/destructor**, **const**, ít nhất **2 toán tử** được đa năng hóa (ví dụ `<<`, `==` hoặc `<` để sắp xếp).
- Dùng **mảng con trỏ đa hình** hoặc lớp template; không rò rỉ bộ nhớ.
- **Lưu và nạp** dữ liệu bằng file văn bản; có **menu** điều khiển.
- Tách tập tin: mỗi lớp một cặp `.h`/`.cpp`, `main.cpp`.

5.3 Sản phẩm nộp và tiêu chí đánh giá

Sản phẩm nộp

- **Báo cáo** (5–10 trang): đặc tả bài toán; bảng phân tích danh từ/động từ; sơ đồ lớp UML; mô tả từng lớp (thuộc tính, phương thức, trách nhiệm); giải thích chỗ dùng kế thừa/đa hình; ảnh chụp kết quả chạy.
- **Mã nguồn** biên dịch được, có chú thích; file dữ liệu mẫu.
- **Bộ ca kiểm thử:** bảng gồm dữ liệu vào, kết quả mong đợi, kết quả thực tế.
- **Trình bày** 10 phút + trả lời câu hỏi; mỗi thành viên giải thích được phần mình làm.

Tiêu chí	Tỷ lệ
Phân tích & thiết kế lớp (UML, quan hệ hợp lý)	25%
Áp dụng đóng gói, kế thừa, đa hình đúng	25%
Chức năng chạy đúng, xử lý lỗi nhập	20%
Lưu/nạp file, quản lý bộ nhớ	10%
Chất lượng mã (đặt tên, tách file, chú thích)	10%
Báo cáo và trình bày	10%

Lộ trình gợi ý

- Tuần 1: đặc tả + sơ đồ lớp, duyệt với giảng viên.
- Tuần 2: cài các lớp, kiểm thử từng lớp.
- Tuần 3: ThuVien/menu, file, hoàn thiện báo cáo.

5.4 Tổng kết môn học

Buổi	Nội dung	Ý tưởng chính
1	Giới thiệu OOP	Đối tượng = dữ liệu + hành vi; đóng gói, kế thừa, đa hình
2–3	Mở rộng của C++	Tham chiếu, new/delete, đa năng hóa hàm, tham số mặc định, inline
4	Lớp và đối tượng	class, private/public, phương thức, phạm vi lớp
5	Constructor/Destructor	Khởi tạo, danh sách khởi tạo, thứ tự gọi, sao chép
6	Thành viên đặc biệt	const, đối tượng thành viên, friend, this, static
7–8	Đa năng hóa toán tử	Toán tử thành viên / friend, `<<` `>>`, `[]`, `=`, ++, sao chép sâu, Quy tắc ba
9	Kế thừa	public/protected/private, gọi constructor lớp cơ sở, ghi đè
10	Đa hình	Hàm ảo, vtable, lớp trừu tượng, hàm hủy ảo
11	Template, nhập/xuất	Lập trình tổng quát; iostream, file văn bản và nhị phân
12	Thiết kế HĐT	Danh từ → lớp, động từ → phương thức, quan hệ, mini-project

Thông điệp cuối

Lập trình hướng đối tượng không chỉ là cú pháp `class`: đó là cách **phân chia trách nhiệm**. Thiết kế tốt giúp **thêm chức năng bằng cách thêm lớp**, thay vì sửa đổi mã đã chạy đúng.

5.5 Bài thực hành buổi 12 – Thiết kế và cài đặt mini-project

Bài 1 – Hoàn thiện chương trình thư viện

- a) Gõ lại và chạy chương trình mục 4; kiểm tra kết quả khớp với slide 4.4.
- b) Thêm lớp `LuanVan` (tên tác giả, năm bảo vệ, mượn tối đa **7** ngày) – xác nhận không phải sửa PhieuMuon.
- c) Thêm hàm `Nap` (mục 4.5) và menu; khởi động chương trình thì nạp `thuvien.txt`, thoát thì lưu.
- d) In danh sách các phiếu **chưa trả** đã quá hạn tại một ngày cho trước, kèm tiền phạt dự kiến.

Bài 2 – Khởi động mini-project của nhóm

- a) Viết đặc tả 10–15 câu cho đề tài đã chọn; lập bảng danh từ → lớp/thuộc tính, động từ → phương thức.
- b) Vẽ sơ đồ lớp UML (kế thừa, thành phần, bội số).
- c) Cài đặt lớp trừu tượng và 2 lớp dẫn xuất đầu tiên, viết `main` thử đa hình qua mảng con trỏ.

5.6 Câu hỏi và bài tập

Câu hỏi

- 1) Trình bày 5 bước thiết kế chương trình hướng đối tượng. Vì sao phân tích danh từ/động từ chỉ cho "ứng viên" lớp?
- 2) Phân biệt kế thừa, thành phần và kết tập; cho ví dụ mỗi loại trong bài toán bán hàng.
- 3) Vì sao TienPhat dùng hàm ảo `SoNgayMuonToiDa()` tốt hơn kiểm tra chuỗi Loai()?
- 4) Vì sao ThuVien phải cấm sao chép (hoặc cài Quy tắc ba)? Điều gì xảy ra nếu viết `ThuVien t2 = tv;` khi cho phép sao chép mặc định?
- 5) Thiết kế (sơ đồ lớp) cho bài toán quản lý bán vé xe khách: tuyến, chuyến, xe, ghế, vé, khách hàng.

Bài tập dự đoán kết quả (có đáp số)

```
TaiLieu *p = new TapChi("T9", "X", 1, 7);
Sach s("S9", "Y", 0, "Z");
PhieuMuon pm(NULL, &s, 3);
cout << p->Loai() << p->SoNgayMuonToiDa() << p->ChoMuonVe() << " "
     << s.ConSach() << " " << pm.TienPhat(17) << " " << pm.TienPhat(20);
delete p;
```

→ **Đáp số:** `TapChi00 0 0 15000` – gọi hàm ảo qua con trỏ cho kết quả của TapChi (0 ngày, false in ra 0); sách số lượng 0 ⇒ ConSach là 0; trả ngày 17: 17 − 3 − 14 = 0 ⇒ không phạt; ngày 20: trễ 3 ngày ⇒ 15000.

- Câu 4: hai đối tượng cùng trỏ tới các TaiLieu ⇒ hàm hủy **delete hai lần** (lỗi double free).